

CONTINUOUS OBSERVABILITY ASSURANCE IN CLOUD-NATIVE APPLICATIONS

PREPRINT, COMPILED MARCH 12, 2025

Maria C. Borges¹ and Sebastian Werner¹

¹Information Systems Engineering, Technische Universität Berlin, Germany

ABSTRACT

When faults occur in microservice applications – as they inevitably do – developers depend on observability data to quickly identify and diagnose the issue. To collect such data, microservices need to be instrumented and the respective infrastructure configured. This task is often underestimated and error-prone, typically relying on many ad-hoc decisions. However, some of these decisions can significantly affect how quickly faults are detected and also impact the cost and performance of the application.

Given its importance, we emphasize the need for a method to guide the observability design process. In this paper, we build on previous work and integrate our observability experiment tool OXN into a novel method for continuous observability assurance. We demonstrate its use and discuss future directions.

Keywords Observability, Microservices, Software Architecture, Continuous Software Engineering

1 INTRODUCTION

Modern cloud-native applications are typically built as microservices, which are developed by different teams, run on complex cloud infrastructure and depend on a diverse software stack [1]. In such dynamic and evolving environments, not everything can be tested upfront, so these applications are typically more prone to faults [2, 3]. When faults occur, developers rely on observability data to quickly identify and diagnose the issue. To collect such data, microservices need to be instrumented and the respective infrastructure configured.

Designing the observability of an application is a challenging task. Different frameworks in the stack expose different data, parameter configurations often depend on the specific tools used, and developers frequently add custom instrumentation code. This process is often underestimated and heavily relies on ad-hoc decisions or “professional intuition” [2, 4]. Poor choices can drive up costs or hinder fault detection, which forces developers into reactive measures, instrumenting and reconfiguring only after problems emerge [5]. Consequently, this results in improvised solutions rather than systematically derived observability designs [3, 5].

As cloud-native microservice applications continuously evolve and improve, so should the observability of the application. Decisions regarding observability must undergo constant reevaluation to ensure continuous improvement and adaptability to changing reliability requirements. Addressing these challenges demands the establishment of a structured process to continuously measure the observability of an application. While most research has typically focused on measuring the cost of observability [6–8], our earlier work introduced metrics and tooling to measure observability more comprehensively [9, 10].

In this paper, we build on that foundation by integrating our previous work into a method for continuous observability assurance. It provides a systematic approach for observability design by incorporating experiment-based assessments proposed in our previous work into the application development lifecycle. It aligns with SRE principles and guidelines [11] to ensure continuous improvement and adaptability to changing reliability trade-offs.

In the following, we show how our method builds upon related industry practices, present the complementary tooling, introduce the method, demonstrate its use and discuss future directions.

2 BACKGROUND: SITE RELIABILITY ENGINEERING

Originating at Google, Site Reliability Engineering (SRE) [11] is a discipline that employs various practices across an organization to assure application reliability. Of particular interest to our work are the practices of error budgets and postmortems. Error budgeting is a practical approach to manage reliability-related trade-offs. By defining an acceptable threshold for system unreliability based on agreed-upon service-level objectives (SLOs), teams can prioritize efforts, balancing between developing new features or conducting reliability improvements. Postmortems are detailed analyses conducted after production incidents. They try to document what happened, why it occurred, and how to prevent recurrence. The goal is to learn from failures to improve systems and processes in the future.

Here, we describe a formalized process for SRE postmortems, derived from Google’s guidelines [11]. Figure 1 illustrates the process.

Step ① - Product Management defines SLOs for a product, which sets an expectation of how much downtime is tolerated for the application per quarter, the so-called “error budget”.

Step ② - Product Engineers develop features as microservices and add observability to the application, including several instrumentation/configuration decisions.

Step ③ - After a period of uptime, the application inevitably runs into a fault and crashes, triggering the incident response process.

Step ④ - Eventually, the incident response team figures out what the root cause of the problem is. Once the emergency has been mitigated, the team starts the process of writing up a postmortem, which captures the incident details, identifies the root cause, describes the actions taken to resolve it, and outlines preventive actions to adopt in the future. This postmortem document serves

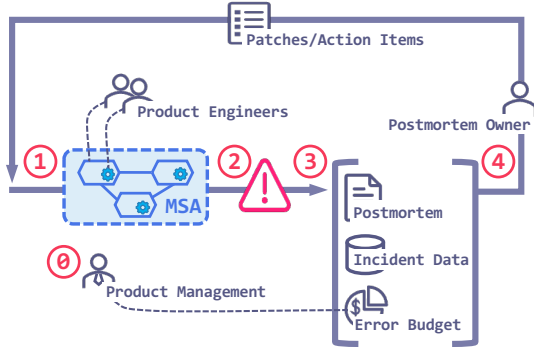


Figure 1: SRE and the Postmortem Cycle

as a reference for future incident prevention and response. It is frequently accompanied by incident data, which can be used for retrospective analysis. Lastly, since the application experienced downtime during the incident, the error budget for the quarter is updated accordingly.

Step ④ - One of the authors of the postmortem document becomes the Postmortem Owner and is accountable for follow-up patches and completion of action items. When these are carried out, the Product Engineer team pushes another update in ① and the cycle begins again.

3 BACKGROUND: OBSERVABILITY EXPERIMENT ENGINE

To assess observability design trade-offs with concrete evidence and arrive at appropriate instrumentation and configuration, we advocate for conducting observability experiments. To facilitate such experimentation, we created OXN [10], a tool that automates this process. With OXN, we can evaluate the observability system's effectiveness by injecting faults into the microservice system under experimentation (SUE) and monitoring the configured detection mechanisms, such as alerting thresholds, while also measuring the costs produced by the observability systems. By observing the system's response to the injected faults, we can assess the effectiveness of the observability system. Figure 2 shows an overview of the tool.

OXN is able to deploy any microservice application specified in a docker-compose file or kubernetes helm chart. After deploying the SUE, OXN starts a workload generator, where load shapes can easily be specified via locust files, a tool commonly used in industry. For the experiments, OXN introduces treatments, which are controlled changes to the system under experiment. We distinguish between fault treatments and instrumentation treatments. OXN already provides a core set of treatments out-of-the-box, which cover some basic use-cases and serve as proof-of-concept. Behind these treatments lies a common interface for by practitioners to implement custom treatments, such as custom fault scenarios. OXN also enables the definition and modification of the SUE's instrumentation, for instance, by adjusting sampling frequencies, adding more instrumentation points, or changing alerting thresholds.

4 METHOD

We present a method for continuous observability engineering. It provides a systematic approach for observability design deci-

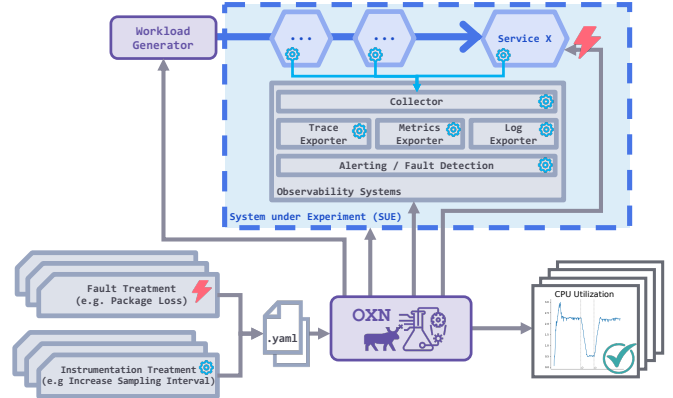


Figure 2: Overview of the tool OXN

sions by incorporating experiment-based assessments and tooling introduced by OXN [10] into the application development lifecycle. Our method follows SRE principles [11] and hooks into the SRE postmortem cycle, promoting continuous improvement and adaptability to changing reliability needs.

Postmortem Observability Assessments

Continuous reliability improvements happen when teams learn from past incidents and take preventive measures. Similar to regression testing, we propose turning past faults into testable incident scenarios. This approach allows teams to spot weaknesses or inefficiencies in the observability machinery and ensures that issues that were once hard to detect are caught early in the future. It also ensures that measures remain in place and hard lessons are not forgotten.

For this purpose, we developed OXN to be extensible and highly customizable to specific application scenarios. Behind each treatment lies a common interface that engineers can leverage to implement custom treatments, for example to simulate a past fault scenario. Here, the open source OXN repository¹ can serve as a platform for engineers to share fault scenarios and instrumentation tuning options across organizations through pull requests. Lastly, a workload generator interface enables the simulation of incident related user behavior. Thus, we propose the following extension to the Postmortem Cycle (see also fig. 3a):

Step ⑤ - The Postmortem Owner extends the OXN fault treatment library to replicate the fault from the recent incident and creates a locust workload for OXN based on the actual incident data. This has two benefits, one the incident becomes testable and thus the patches and system improvement apparent, second, these experiments can be re-run to continuously verify that future modifications don't reintroduce issues, thus, increasing system reliability in the long term.

Continuous Observability Assurance

In this section, we explain how observability experiments can be employed to continuously improve the observability of microservice applications. Figure 3b illustrates this process.

¹<https://github.com/nymphbox/oxn>

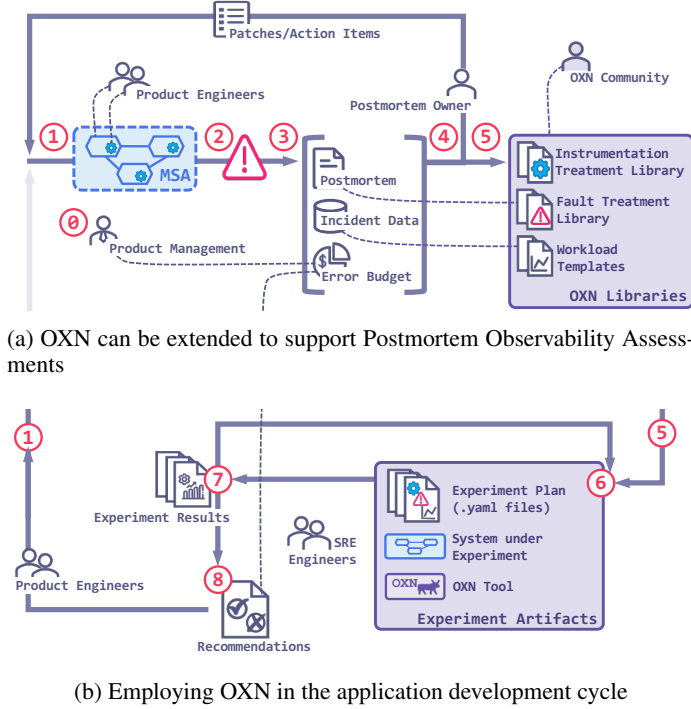


Figure 3: Method for Continuous Observability Assurance

Step ⑥ - The SRE Engineers identify experiments of interest. For each experiment, they create an experiment plan, i.e. .yaml files with a machine readable description of the experiment design, including what to observe, what instrumentation to tweak, what faults to inject, and what load to use. With the rest of the experiment artifacts, they can run experiments.

Step ⑦ - The SRE Engineers run several experiments and collect results showing how effective different observability configurations are at identifying the faults relevant to the application and how costly they are.

Step ⑧ - The SRE Engineers identify and evaluate trade-offs. Based on how much downtime can be tolerated in the rest of the quarter (i.e. what is left in the error budget), they might make different recommendations to the Product Engineering team. Both teams negotiate and decide on implementation changes for the next release, completing the feedback loop in the next step (①).

This method enables a collaborative and systematic design for observability, which can be continuously applied during the development of cloud-native applications. It empowers the broader community to build a reproducible repository of fault scenarios, which can be automatically tested using OXN. Furthermore, its integration into the SRE lifecycle allows for effective control over when, how often, and at what cost these observability design decisions need to be updated or refined.

5 PRELIMINARY INSTANTIATION

In this section, we present a preliminary demonstration of our method, while acknowledging the need for further validation, e.g. through collaboration with industry SRE teams. For the demonstration, we construct an example based on a common fault. Specifically, the incident involves a latency spike in a dependent

Table 1: Cost of the Design Alternatives

	Baseline	A	B
CPU time	191.83s	197.68s	202.06s
Overhead	-	+3.05%	+5.33%

service, a recurring issue documented in both research [12] and industry blogs [13].

As our microservice application, we use the OpenTelemetry Astronomy Shop Demo². It consists of 20 microservices and is instrumented to collect several metrics and traces across the whole application, making it a suitable candidate to demonstrate the method. We deploy the application on a cloud-based virtual machine (8vCPUs, 32GB Memory).

Since we lack real incident data for experimentation, we simulate a constant load of 50 concurrent users. For instrumentation, our focus is on determining an appropriate sampling rate for tracing data—a configuration option already supported by the OXN tool. To model a latency spike incident in step 5, we use a network delay fault injection mechanism supported by OXN. This feature allows us to introduce a configurable network delay between two services, which in our case we configure to range between 0-90ms. As our experiment plan (step 6), we run experiments for 10 minutes each, with 4 minutes allocated for ramp-up, 2 minutes for the fault occurrence, and the final 4 minutes for a return to normal conditions. For more details, consult our delay experiments in the experiment plan³.

For step 7, we collect results for trace duration across three different tracing sampling rates, 1%, 5% and 10%, as shown in Figure 4. The fault is hardly recognizable in the baseline variant, however, it becomes more visible in the two configuration alternatives A and B. The graph shows a slight increase during the fault’s duration, an increase also detected by the simple fault classifier implemented in OXN. Table 1 shows an overview of the costs for each configuration option, measured as CPU seconds across the relevant services.

Through the conducted observability experiments, practitioners can see that the fault modeled in our example becomes more visible by increasing the tracing sampling rate. This, of course, comes at a cost. By revisiting step 6, further experiments could explore new metrics to identify the fault at a lower cost. Practitioners would then make recommendations based on the available error budget in step 8. With this demonstration, we hope to introduce the tooling to practitioners and invite them to explore, refine and apply the method in practice.

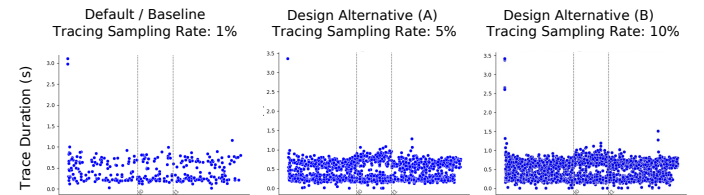


Figure 4: Comparison of Trace Duration Results for Fault Treatment NetworkDelay[0-90ms]

²<https://github.com/open-telemetry/opentelemetry-demo>

³<https://github.com/nymphbox/oxn/tree/main/experiments>

6 RELATED WORK

Observability has been evaluated by means of experimentation several times in the past, though these assessments have predominantly focused on the costs associated with observability. For instance, Reichelt et al. [7] conducted a concrete benchmark of observability instrumentation. Their study introduced a tool for continuous measurement of overhead of popular instrumentation libraries. More recently, Dinga et al. [8] experimentally investigated the energy efficiency of different observability tools. In another experiment-based study, Ahmed et al. [14] explored the effectiveness of four monitoring tools in identifying performance regressions but assumed the default configuration for every tool, therefore ignoring instrumentation and configuration decisions. While all of the three papers investigate an important slice of the observability design landscape, a comprehensive method for continuous observability assurance is still missing.

Beyond experimentation, Srinivas et al. [5] suggest a method for selecting metrics for microservices based on service properties. The paper offers a path to navigate the observability design space, but does not tackle the problem of configuration once the metrics have been selected. Additionally, their approach offers a fixed selection of metrics for each service, which neglects the continuous nature of service development and chances for improvement after production incidents. With the method introduced in our paper, we can continuously validate these metric selections and adjust their configuration for higher fault visibility during incidents.

7 CONCLUSION

In this paper, we propose a method that makes observability design decisions testable and an integral part of continuous software engineering. We outline concrete steps in the application development lifecycle, and align these processes with the SRE post-mortem cycle [11], incorporating tooling provided by OXN [10].

Additionally, we argue for a community-driven approach, which can be continuously applied during the development of cloud-native applications. Moreover, by rooting our methods in the SRE lifecycle, we benefit from error budgeting, which allows practitioners to balance reliability improvements and feature development, giving a systematic framework for applying OXN effectively.

For future work, we aim to validate and refine this method by applying it to different postmortems in opensource application projects. Besides that, we continue to extend the OXN tool with features to foster broader adoption of experiment-driven observability assurance.

ACKNOWLEDGEMENTS

Funded by the European Union (TEADAL, 101070186). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

- [1] J. Bogner, J. Fritzsche, S. Wagner, and A. Zimmermann, “Microservices in industry: Insights into technologies, characteristics, and software quality,” in *2019 IEEE Intl. Conf. on Software Architecture Companion (ICSA-C)*, 2019, pp. 187–195.
- [2] H. Zhang, S. Li, Z. Jia, C. Zhong, and C. Zhang, “Microservice architecture in reality: An industrial inquiry,” in *Intl. Conf. on Software Architecture*, 2019, pp. 51–60.
- [3] S. Niedermaier, F. Koetter, A. Freymann, and S. Wagner, “On observability and monitoring of distributed systems – an industry interview study,” in *Intl. Conf. on Service-Oriented Computing*, 2019, pp. 36–52.
- [4] G. Vale, F. F. Correia, E. M. Guerra, T. de Oliveira Rosa, J. Fritzsche, and J. Bogner, “Designing microservice systems using patterns: An empirical study on quality trade-offs,” in *Intl. Conf. on Software Architecture*, 2022, pp. 69–79.
- [5] P. Srinivas, F. Husain, A. Parayil, A. Choure, C. Bansal, and S. Rajmohan, “Intelligent monitoring framework for cloud services: A data-driven approach,” in *46th Intl. Conf. on Software Engineering: Software Engineering in Practice*, 2024, p. 381–391.
- [6] D. Ernst and S. Tai, “Offline trace generation for microservice observability,” in *IEEE 25th Intl. Enterprise Distributed Object Computing Workshop*, 2021, pp. 308–317.
- [7] D. G. Reichelt, S. Kühne, and W. Hasselbring, “Overhead comparison of opentelemetry, inspectit and kieker,” in *Symp. on Software Performance*, 2021.
- [8] M. Dinga, I. Malavolta, L. Giamattei, A. Guerriero, and R. Pietrantuono, “An empirical evaluation of the energy and performance overhead of monitoring tools on docker-based systems,” in *Intl. Conf. on Service-Oriented Computing*, 2023, pp. 181–196.
- [9] M. C. Borges, J. Bauer, S. Werner, M. Gebauer, and S. Tai, “Informed and assessable observability design decisions in cloud-native microservice applications,” in *2024 IEEE Intl. Conf. on Software Architecture (ICSA)*, 2024, pp. 69–78.
- [10] M. C. Borges, J. Bauer, and S. Werner, “Oxn - automated observability assessments for cloud-native applications,” in *2024 IEEE Intl. Conf. on Software Architecture Companion (ICSA-C)*, 2024, pp. 167–170.
- [11] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google runs Production Systems*. O’Reilly Media, Inc., 2016.
- [12] S. Ghosh, M. Shetty, C. Bansal, and S. Nath, “How to fight production incidents? an empirical study on a large-scale cloud service,” in *13th Symp. on Cloud Computing (SoCC’22)*, 2022, p. 126–141.
- [13] M. Smiley, “How we diagnosed and resolved redis latency spikes with bpf and other tools,” 2022. [Online]. Available: <https://about.gitlab.com/blog/2022/11/28/how-we-diagnosed-and-resolved-redis-latency-spikes/>
- [14] T. M. Ahmed, C.-P. Bezemer, T.-H. Chen, A. E. Hassan, and W. Shang, “Studying the effectiveness of application performance management (apm) tools for detecting performance regressions for web applications: An experience report,” in *Intl. Conf. on Mining Software Repositories*, 2016, p. 1–12.